

# 计算机组成原理·知识手册 13 张卡片·约 45 分

## 一、性能指标

高频·必考 / 计算·约 6 分钟

### 主频、CPI、CPU 时间三件套

#### 核心公式

- CPU 时钟周期 =  $1 / \text{主频}$
- CPU 时钟周期数 =  $\Sigma(\text{指令条数} \times \text{该类指令的 CPI})$
- CPU 执行时间 =  $\text{指令条数} \times \text{CPI} \times \text{时钟周期} = \text{指令条数} \times \text{CPI} / \text{主频}$
- MIPS =  $\text{指令条数} / (\text{执行时间} \times 10^6) = \text{主频} / (\text{CPI} \times 10^6)$
- FLOPS = 每秒浮点运算次数 (GFLOPS=10<sup>9</sup>)

#### 存取与带宽

- 存取时间 Ta: 发出读命令到数据到手; 存取周期 Tc: 连续两次独立访问的最小间隔,  $T_c > T_a$  (要等恢复)。
- 总线带宽(MB/s) =  $\text{总线数据宽度}(\text{bit}) / 8 \times \text{总线工作频率}(\text{MHz}) \times \text{每周期传输次数}$  (DDR 为 2)。
- 磁盘: 数据传输率 = 记录密度  $\times$  磁盘转速对应的线速度; 平均寻道时间 + 旋转延迟 (= 一圈时间/2) + 传输时间。

#### 易错

1. 「主频高就一定快」错: CPI 与指令条数同样决定时间; 不同机器比较要用同一基准程序。
2. 平均 CPI 要按各指令的条数占比加权, 不是按时间占比。
3. GMIPS/加权平均在不同程序规模下会给出不同结论, 题目常问「哪种说法正确」。

▷ 计算题先统一单位 (GHz→MHz→Hz; ns→s), 把公式抄在草稿第一行, 避免最后一步量级错。

## 二、数据表示与运算

高频·必考 / 计算·约 6 分钟

### 原/反/补/移码与溢出判断

表示范围 (n 位, 含 1 位符号)

- 原码:  $-(2^{n-1}-1) \sim +(2^{n-1}-1)$ , 0 有两种表示
- 反码: 同上, 0 有两种表示
- 补码:  $-2^{n-1} \sim +(2^{n-1}-1)$ , 0 唯一, 多出最小的那个负数 ( $1000\cdots0 = -2^{n-1}$ )
- 移码: 与补码仅符号位相反, 0 唯一; 只用来表示浮点数的阶码 (便于比较大小)

转换: 负数原 $\rightarrow$ 反 (符号不变, 数值取反)  $\rightarrow$ 补 (末位 +1)。已知补码求原码: 再取反加一。符号扩展: 补码用符号位扩展, 原码/反码在数值位两侧补 0; 无符号数左移补 0。

### 溢出判断

- 双符号位 (变形补码): 结果符号  $01 \rightarrow$  正溢,  $10 \rightarrow$  负溢,  $00/11$  无溢出。
- 单符号位:  $C_f \oplus C_n = 1$  (符号位进位与最高数值位进位不同) 即溢出。
- 只有同号相加才可能溢出; 异号相加永不溢出。左移可能溢出 (丢高位), 右移只会损失精度 (算术右移补符号位, 逻辑右移补 0)。

标志位 (无符号与有符号共用同一加法结果, 差别只看 flags)

- ZF 零、SF 最高位、CF 无符号进/借位、OF 有符号溢出。
- `cmp a,b` 后: 无符号比大小看 CF/ZF; 有符号看  $SF \oplus OF$  与 ZF。

IEEE 754 浮点数：格式、范围、特殊值

| 项    | 单精度 float                              | 双精度 double         |
|------|----------------------------------------|--------------------|
| 位数   | $1 + 8 + 23 = 32$                      | $1 + 11 + 52 = 64$ |
| 阶码偏置 | 127                                    | 1023               |
| 真值   | $(-1)^s \times 1.M \times 2^{(E-127)}$ | 同理，偏置 1023         |

- 尾数隐藏最高位的 1（规格化后整数位恒为 1，不必存），所以 23 位尾数实际有 24 位精度。
- 阶码全 0：表示  $\pm 0$  及非规格化数（隐含整数位为 0，阶码真值取  $1 - \text{偏置} = -126$ ），用来表示接近 0 的数。
- 阶码全 1：尾数为 0  $\rightarrow \pm\infty$ ；尾数非 0  $\rightarrow \text{NaN}$ 。
- 表示范围：单精度约  $\pm 1.2 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$ ；精度约 7 位十进制有效数字（双精度 15~16 位）。
- 十进制转 754：先转二进制  $\rightarrow$  规格化  $\rightarrow$  阶码加偏置  $\rightarrow$  尾数舍入（0 舍 1 入）。

常考比较

- 浮点数加减要先对阶，小阶向大阶靠拢（尾数右移，可能丢精度）。
- 左规/右规：结果非规格化要左规（尾数左移、阶码减 1）；溢出（两正数相加导致最高位为 1）要右规（阶码 +1）。
- 浮点数 0 有两种表示（+0/-0）；「-0」在 754 里有编码。
- 与整数比较时注意：0.1 无法用二进制精确表示， $0.1 + 0.2 \neq 0.3$  的根源就在尾数位数有限。

## 加法器、ALU 与校验码

### 加法器

- 串行进位：进位逐级传递，慢。
- 并行进位（超前进位 CLA）：由  $G_i = A_i B_i$ 、 $P_i = A_i + B_i$  直接生成各级进位，用面积换速度。
- ALU 由加法器 + 辅助逻辑构成，做加减与逻辑运算；减法 = 加减数的补码。
- 原码一位乘/补码 Booth 乘法、恢复余数/不恢复余数除法：408 里只考原理与位数，不要求手算流程。

### 校验码（每年必有一题）

- 奇偶校验：+1 位，只能检错不能纠错，检测奇数个错。
- 海明码：设校验位  $r$  位、信息位  $n$  位，需满足  $2^r \geq n + r + 1$ 。校验位放在 2 的幂次位置（1, 2, 4, 8...），每个校验位按对应位组的奇偶生成；纠错时把各校验组的结果拼成指误码（指出错在哪一位）。海明码可纠 1 位错、检 2 位错（再加总奇偶位）。
- CRC 循环冗余校验：数据后附  $r$  个 0，模 2 除以生成多项式  $G(x)$ ，余数即校验位（ $G$  最高次为  $r$ ，即  $G$  有  $r+1$  位）。CRC 用异或、不进位的模 2 运算，能检测突发错误，硬件用移位寄存器实现。

▷ 问「哪种编码能纠错」：海明码。问「检错能力最强/常用于磁盘与网络」：CRC。

### 三、存储系统

高频 · 必考 / 计算 · 约 6 分钟

#### 层次结构与主存容量计算

层次：寄存器 → Cache → 主存 → 外存。依据是程序访问的局部性（时间局部性：刚用过还会再用；空间局部性：邻近的会被用）。目标：容量接近外存、速度接近寄存器、成本接近外存。

主存指标：MAR 位数决定可寻址单元数 ( $2^{\text{MAR}}$ )，MDR 位数 = 存储字长；容量 =  $2^{\text{MAR}} \times (\text{MDR}/8)$  字节。按字节编址时 MAR=32 → 寻址空间 4GB。

#### SRAM vs DRAM

|             | SRAM    | DRAM                         |
|-------------|---------|------------------------------|
| 元件          | 触发器     | 电容 + 单管                      |
| 刷新          | 不需要     | 需要（行地址选中后按行刷新）               |
| 速度/集成度/功耗/价 | 快、低、大、高 | 慢、高、小、低                      |
| 用途          | Cache   | 主存                           |
| 地址引脚        | 行列分开    | 行列地址复用，引脚数减半（所以地址线数是容量位数的一半） |

刷新方式：集中（有死区、控制简单）、分散（无死区但速度减半）、异步（结合两者，实际常用）。

芯片扩展：位扩展（加数据宽度，不加地址）、字扩展（加容量、需片选与译码器）、字位同时扩展。所需芯片数 = (总容量)/(单片容量)，地址线看总寻址范围、片选用高位地址经 3-8 译码器。

## Cache：映射、地址划分、平均访问时间

### 三种映射（把主存块放进 Cache 的规则）

- 直接映射：块号  $\bmod$  Cache 行数  $\rightarrow$  地址 = 标记 Tag | 行号 | 块内地址。冲突最多、硬件最简。
- 全相联：任意行，地址 = Tag | 块内地址；比较电路最贵，冲突最少。
- 组相联：组号 = 块号  $\bmod$  组数  $\rightarrow$  地址 = Tag | 组号 | 块内地址。n 路组相联 = 每组 n 行。

算地址位数的固定步骤：块内地址 =  $\log_2(\text{块大小})$ ；组号位数 =  $\log_2(\text{组数})$ ；剩下的是 Tag。行数 = Cache 容量 / 块大小；组数 = 行数 / 路数。

### 命中率与时间

- 先访 Cache，不命中再访主存： $T_a = h \cdot T_c + (1-h) \cdot T_m$ （题若说「同时访问」则取 max，按题意来）。
- $h = \text{命中次数} / (\text{命中} + \text{不命中})$ 。缺失惩罚 = 把块取进 Cache 的时间。
- 局部性越好、Cache 越大  $\rightarrow h$  越高，但边际收益递减；块太大时，块内局部性提升但缺失惩罚与冲突 miss 上升。

替换算法：FIFO、LRU（栈式，需计数/堆栈实现，命中率高）、随机法（易实现）。不能替换刚进来的块（随机法例外）。写策略：全写法（写 Through，Cache 与主存一致，配合写缓冲）；写回法（只改 Cache，换出时才写回，需脏位）。写不命中：写分配法（调块，常配写回）、非写分配法（直接写主存，常配全写）。

易错：Cache 与主存分块大小相同；标记 + 有效位 + 脏位也要算进 Cache 的总容量；「Cache 命中率 100%」只在理想循环且不考虑冷启动时成立。

## 虚拟存储器：页表、TLB 与地址转换

**要点：**虚存解决的是容量（用外存把主存扩大），Cache 解决的是速度。虚实地址转换由硬件 + 操作系统共同完成，对用户程序透明。

- **页式：**页大小  $2^p$ ，虚地址 = 虚页号 | 页内偏移；页表存「页号 → 物理页框号 + 有效位 + 修改位 + 访问位…」。
- **段式：**段号 | 段内偏移，段长可变，便于共享与保护，但会产生外部碎片。
- **段页式：**先分段再分页。

### 一次访存的真实过程

1. **查 TLB（快表）：**命中 → 拿到页框号 → 拼物理地址 → 再查 Cache/主存。
2. **TLB 未命中 → 查页表（可能多次访存，多级页表）；**页不在主存 → 缺页中断，从磁盘调入，可能触发页面置换。
3. **最坏情况一次逻辑访存要访问：**页表 + 主存数据（+ 磁盘）。TLB 的存在就是为了把「查页表」的开销摊薄。

### 必须分清三类「不命中」

- **Cache 不命中 → 访问主存，对程序透明，由硬件处理。**
- **TLB 不命中 → 查页表，仍是硬件/OS 协作。**
- **缺页 → 必须访问磁盘，代价最高（毫秒级），且需要 OS 介入与页面置换。**

**置换算法：**OPT（不可实现，作基准）、FIFO（会出现 Belady 异常）、LRU（常用，需硬件计数）、Clock/NRU（近似 LRU，用访问位循环扫描）。

## 四、指令系统

高频 · 必考 / 计算 · 约 6 分钟

### 指令格式与扩展操作码

指令 = 操作码 OP + 地址码。三地址指令  $A_1 = A_2 \text{ op } A_3$ ；零地址指令用于停机、空操作或堆栈运算（操作数在栈顶）。

扩展操作码算法（每年必考型）：给定指令字长与各地址字段长度，从四地址→三地址→…→零地址逐级扩展：

- 每级可用的操作码个数 = 上一级剩余前缀数  $\times 2^{\text{新增字段位数}}$ ；
- 该级要留出的编码数 = 本级指令条数，剩下的才允许进入下一级。
- 铁律：短码不能是长码的前缀。

例：16 位指令、地址码各 4 位，若有 15 条四地址、14 条三地址，则四地址用完 15 个前缀、留 1 个进入下一级 → 三地址可用  $1 \times 2^4 = 16$  个编码，用掉 14 剩 2……逐级算下去即可求「零地址最多几条」。

### 寻址方式与有效地址 EA

| 方式    | EA                 | 访存次数（取操作数）                   |
|-------|--------------------|------------------------------|
| 立即    | 操作数在指令里            | 0                            |
| 直接    | $EA = A$           | 1                            |
| 一次间接  | $EA = (A)$         | 2                            |
| 寄存器   | $EA = \text{寄存器号}$ | 0                            |
| 寄存器间接 | $EA = (R)$         | 0（访存 1 次）                    |
| 变址    | $EA = (IX) + A$    | 1（适合数组）                      |
| 基址    | $EA = (BR) + A$    | 1（扩大寻址、多道程序）                 |
| 相对    | $EA = (PC) + A$    | 1（PC 是下条指令地址，转移范围有限但代码浮动可执行） |

CISC vs RISC：RISC 定长指令、只有 Load/Store 访存、寄存器多、硬布线控制器、高度流水线化；CISC 指令功能强、变长、微程序控制、便于高级语言映射。

## 五、CPU

高频 · 必考 / 选择 · 约 6 分钟

### 指令周期、数据通路与控制

时间层次：时钟周期（CPU 周期）< 机器周期（总线周期，取指/读内存等）< 指令周期（执行完一条指令）。一条指令的指令周期 = 取指 + 间址 + 执行 + （中断），其中间址周期与中断周期不一定有，但任何指令都必须有取指周期和执行周期。

取指周期公干（每章指令都一样）： $(PC) \rightarrow MAR$ 、 $M(MAR) \rightarrow MDR$ 、 $M(MDR) \rightarrow IR$ 、 $(PC)+1 \rightarrow PC$ （宽度取决于指令字节数）。

- 指令寄存器 IR 保存当前指令；操作码译码器产生控制信号；状态寄存器（PSW/flags）记录结果特征。
- 通用寄存器组既可给程序员用，也参与运算；专用寄存器（PC、IR、MAR、MDR、PSW）程序员不可见或只可读。
- 单总线 CPU：ALU 两个输入若共用一条总线，需要暂存器 Y、Z（或锁存器）来避免冲突；双/三总线可减少冲突但硬件更贵。

### 两类控制器

- 硬布线（组合逻辑）：速度快、RISC 用；设计难、扩充不易。
- 微程序：机器指令对应一段微程序，微指令存放在控制存储器 CM（ROM）里；一条微指令可同时发出多个不冲突的微命令；微程序入口地址由操作码映射（或字段直接编码）。水平型微指令并行性强、字段长；垂直型接近机器指令、并行性差。

中断响应时机：在每条指令执行周期结束时统一检查有无中断请求，条件为：有请求 + 开中断 + 非总线周期禁止状态。

## 流水线：公式、三种相关、性能瓶颈

### 基本公式 (k 个功能段、n 个任务)

- 流水线周期  $\Delta t$  = 最长段的时间 (各段不等长时按最长段统一, 多余时间空闲)。
- 执行时间  $T = (k + n - 1) \times \Delta t$ ; 理想顺序执行  $T_0 = n \times k \times \Delta t$ 。
- 吞吐率  $TP = n / T = 1 / \Delta t \times n / (k + n - 1)$ ,  $n \rightarrow \infty$  时趋于  $1 / \Delta t$ 。
- 加速比  $S = T_0 / T = nk / (k + n - 1) \rightarrow k$ 。
- 效率  $E = \text{加速比} / k = n / (k + n - 1)$ 。

### 三类相关 (阻塞的原因)

1. 结构相关: 资源冲突 (如取指与访存都要用内存)  $\rightarrow$  存指分离 (哈佛结构) 或插气泡。
2. 数据相关: 后一条要用前一条的结果 (RAW 最常见)  $\rightarrow$  转发/旁路 (forwarding) 解决大部分, 仍不能完全消除时插入暂停周期 (load-use 相关)。
3. 控制相关: 转移指令使后面取指无效  $\rightarrow$  分支预测、提前形成条件码、延迟分支 (转移指令后放一条必执行指令)。

### 必须会答的追问

- 「段数越多越快吗」不: 超过一定数量后, 锁存器开销与阻塞比例上升, 反而变慢。
- 「乱序执行/超长指令字」属于更高级的性能手段, 408 只到概念层面。
- 计算题里若给了各段延迟与锁存开销, 先算  $\Delta t$  再套公式, 别把  $\Delta t$  当成平均段长。

## 六、总线

选择 / 计算 · 约 6 分钟

### 总线类型、仲裁与性能

按位置/功能：片内总线、系统总线（数据总线/地址总线/控制总线）、通信总线。

- 数据总线宽度决定一次传输多少位；地址总线宽度决定寻址范围；控制总线传命令、状态、时序。
- 三态门（高电平/低电平/高阻）+ 总线事务（传输命令 + 传输数据），同一时刻只允许一个主设备发送（分时共享、单向或双向传输）。

### 仲裁（谁获得总线控制权）

- 链式查询：只需一根 BS 线，简单但对靠近控制器的设备优先、对电路故障敏感。
- 计数器定时查询：优先级可设置，线数多 ( $\lceil \log_2 n \rceil$ )。
- 独立请求：每个设备一对 BR/BG 线，响应最快、线数最多、控制最灵活。

时序：同步总线（公共时钟，快、要求设备速度匹配）、异步总线（握手：不互锁/半互锁/全互锁）、半同步（加 WAIT 线）。带宽：= 数据宽度/8 × 工作频率 × 每时钟传输次数；注意「地址传输占用的周期」和「突发传输」这两处常设的坑。

标准常识：PCI/PCIe（串行、点对点、lanes 聚合）、USB（主从、热插拔）、SATA、SPI/I<sup>2</sup>C（片内/板级）、ISA（并行、已淘汰）。

## 七、输入输出

高频·必考 / 大题·约 6 分钟

### 程序查询、中断、DMA 三者对比

| 方式         | CPU 与设备       | 数据传输由谁做         | CPU 开销          | 适用         |
|------------|---------------|-----------------|-----------------|------------|
| 程序直接控制（查询） | 串行，CPU 轮询状态位  | CPU             | 一直等，浪费极大        | 简单系统       |
| 程序中断       | 并行，设备好了通知 CPU | CPU（保存现场、逐个字搬运） | 每次中断有额外开销       | 字符设备、键盘    |
| DMA        | 并行            | DMA 控制器直接访存成组传送 | 只在块开始/结束时中断 CPU | 磁盘、显卡等高速大块 |
| 通道         | 并行            | 通道执行通道程序        | 更独立             | 大型机、多设备并发  |

中断处理流程：请求 → 判优（优先级） → 响应（关中断、保存断点与现场） → 服务程序 → 恢复现场 → 开中断 → 返回。

- 中断向量 = 中断服务程序的入口地址（中断向量表按中断类型号索引）。
- 允许多重中断（中断嵌套）的前提：响应中断后及时开中断；屏蔽字用来动态改变优先级。
- 用户程序里「管态/目态」：特权指令只能管态执行（改 PSW、启动设备、置时钟等），目态执行会触发访管中断（系统调用）。

DMA：需要总线使用权（周期窃取 / 停止 CPU 访问内存 / CPU 与 DMA 交替使用内存三种），传送单位是数据块；预置：内存首地址、传送字数、传送方向；结束后 DMA 向 CPU 发一次中断。DMA 与 CPU 争夺主存是必然的，靠周期窃取解决。

I/O 编址：统一编址（把端口当内存地址，用访存指令，占用内存空间）；独立编址（专门的 IN/OUT 指令 + 独立地址空间，控制线多但程序清晰）。

## 八、大题套路

高频·大题 / 计算·约 6 分钟

### 组成原理计算题的四种固定解法

#### 1. 地址位数划分题 (Cache/虚存/TLB 混在一起考)

步骤：块内地址= $\log_2$ (块长)  $\rightarrow$  组号/行号= $\log_2$ (组数)  $\rightarrow$  Tag = 主存地址位数 - 前两者。物理地址再按同样方式切一次；两套低位字段位数相同是判断「是否同一块」的依据。

#### 1. 容量与芯片扩展题

先写总寻址 ( $2^{\text{地址线}}$ ) 与总位宽 (数据线)，再算单片容量  $\rightarrow$  位数比得位扩展数、地址比得字扩展数；别忘了行列地址复用会让 DRAM 引脚数减半。

#### 1. 流水线时间题

先求  $\Delta t$  (含锁存开销)，再数「实际有多少条指令、第几条被阻塞几拍」，公式  $T=(k+n-1)\Delta t$  只适用于无阻塞；有阻塞时把气泡拍数逐个加上。

#### 1. 指令/微指令格式题

按「总位数 = 各字段之和」列方程：操作码位数、控制字段数、下地址字段位数、直接/字段译码方式。水平型微指令的微命令位数就是控制字段宽度，字段间接编译法用「扩展标志位」占 1 位。

通用作答习惯：单位、位数、地址边界三样东西先抄到草稿；最后一步把结果代回原题验算位数之和，这一遍能救回一半的错。

---

计算题密度最大：地址划分、Cache、流水线、扩片。公式与位数划分是主要失分点。

「考频」是按题型惯例给出的人工判断，不是官方统计；大纲与分值以当年《考试大纲》《考试分析》为准。

来源：本地手写（依据 408 考试大纲与通行教材整理，不复制任何第三方受版权保护的讲义）；构建于 2026-09-28 10:15。公式渲染失败的条目会以红色等宽字体保留原文。